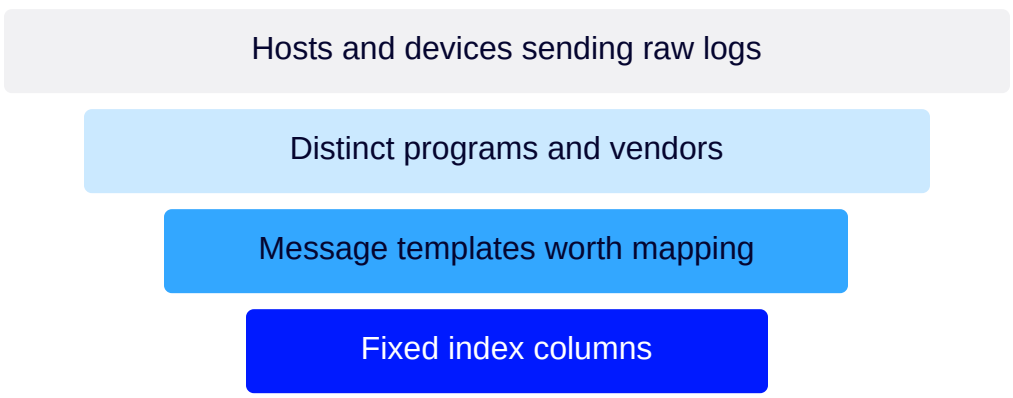
Scaling to hundreds of raw log sources

Maintain templates and a generator, not queries. Effort grows with formats, not hosts.

EFFORT FOLLOWS FORMATS, NOT HOSTS



Widths show direction, not measured counts.

GROUP SOURCES BY GRAMMAR

`Syslog`	ProcessName
`CommonSecurityLog`	DeviceVendor + DeviceProduct
`Custom _CL with RawData`	Source tag or first token of the line
`Custom _CL with JSON`	Schema fingerprint: sorted list of keys

500 web servers running the same sshd share one set of templates.

SHRINK THE UNMATCHED SHARE EVERY WEEK

Unmatched lines
Rows that only reached value regex

Mine templates
Notebook runs Drain3 per grammar group

Rank by volume
Largest new templates first

Analyst maps fields
Approves the template and its ASIM fields

Generator rebuilds jobs
KQL jobs regenerated from Git

Repeat: each pass moves the largest remaining formats from Tier 2 to Tier 1

WHERE RAW LINES LAND, PASS BY PASS



WHAT RUNS WHERE

Template mining	Spark notebook	Weekly
Index build	One KQL job per grammar group	Hourly or daily
History backfill	Spark notebook	On demand
Coverage report	KQL job or notebook	Each build
Hunting	Entity index, interactive or MCP	Any time

RULES THAT KEEP IT FAST

- Filter on the grouping key and a `has` term before any regex.
- One job per grammar group, never one per host.
- Unmatched lines always go to value regex, so nothing is dropped.
- Stagger job schedules to stay inside the tenant job quota.
- Rebuild jobs from the registry; never hand-edit them.

Sources: Microsoft Learn, data lake KQL jobs and notebooks; LevelBlue design

Checked 16 October 2026

© 2026 LevelBlue Intellectual Property. LevelBlue logo, and registered trademarks and service marks are the property of LevelBlue Intellectual Property and/or LevelBlue affiliated companies. All other marks are the property of their respective owners.